STRUCTURE OF AN YACC SCRIPT

A template of a yacc script is given here. Note that the block delimited by "%{" and "%}" are meant for C declarations similar to a lex script. After "%}" and before "%%", the space is used for declarations specific to yacc. These include, among others,

- Declarations enclosed within a "%union { }" block, that are used to define attributes of grammar symbols, both terminals and nonterminals. The attributes of terminals are defined and supplied by the scanner, while the attributes of the non-terminals are defined and used by the parser.
- One or more lines beginning with "%token" to declare the terminals that both the scripts intend to share.
- One or more lines beginning with %type to define the attributes of the nonterminals that are used exclusively by the parser.
- There are other forms of % declarations used by the parser.

In the space between the two "%%" delimiters, the production rules of the CFG are written. The first nonterminal is the start symbol by default. A production rule is written by giving the lhs nonterminal followed by ":", and the different alternates are separated by "|". The end of all rules for the same nonterminal is indicated by a ";".

After the 2nd "%%", the definition of a yyerror() function and a main() are required. The parser, yyparse(), which is called by main(), in turn calls the yyerror() function when its encounters a syntax error.

```
/*C declarations*/
%{
  #include <stdio.h>     // required for i/o
  int yylex(void);         // prototype of yylex() which is called by yyparse()
  void yyerror(char *); // yyparse() calls a function named as yyerror() on a  syntax error
  extern int yychar, yylineno;      // used by yyerror()
%}
/* YACC Declarations */
%union { ….. }
%token ……
%type  ……...
/* Grammar follows */
%%
A :  α          {/* for a rule A → α | β  and to add action in C */}
   |  β
   ;
……
%%
void yyerror(char *s)
  {printf(" %s  near symbol %c \n", s, (char)yychar);}
```

```
int main ()  {yyparse (); return 0;}
```

==========

In order for a yacc script to do useful parsing in conjunction with its partner lex script, the following. Points may be noted.

- One means of communication between yylex() and yyparse(), is via the terminal symbols, that are defined as tokens. Define the tokens in the yacc script, using "%token" directive and use yacc –d to create the file "y.tab.h". Include "y.tab.h" in the lex script; return the tokens by their defined name in the action part of the regex in the lex script. The actual communication are through the values saved in the "y.tab.h" file. Such values can be seen in the parser output by displaying the token name in the yacc action.
- Another method of communication is by using a single character. Single characters have to be returned in the lex script and these characters are placed in the grammar rules in the same form. It can be seen that the ASCII values are actually passed around between the 2 tools.
- The lex variable, yytext, can also be made accessible in the parser by declaring it as an external variable, however this way of communication has limited use.

End of Document